

SafeVoice Scanner

Suspicious Link and QR Code Checker for Visually Impaired Users

Graduation Project Report

Prepared by:

Name	ID
Munira Alajalin	446007593
Turfa Aldulaimi	446005451
Rana Aleisa	445014255
Yasmeen Alrashidi	446015511
Nawal Ahmed Awaji	446005484

Supervised by:

Dr. Areej Alghaith

Imam Mohammad Ibn Saud Islamic University
Applied College

Contents

Chapter 1: Introduction	5
1.1 Introduction.....	6
1.2 Problem Definition.....	6
1.3 Proposed Solution	7
1.4 System Scope	8
1.5 Goals and Objectives	8
1.6 Hardware & Software Requirements	9
1.7 System Cost	9
1.8 Timeline	10
1.9 Roles & Responsibilities.....	12
Chapter 2: Background & Literature Review	13
2.1 Introduction.....	14
2.2 Background	14
2.3 Related Work	15
2.4 Competitive Analysis.....	18
Chapter 3: System Methodology & Analysis	20
3.1 Introduction.....	21
3.2 System Methodology	21
3.3 User Characteristics	23
3.4 User Requirements.....	23
3.5 Software Requirements and Specifications	24
Chapter 4: Cybersecurity Threat Landscape	26
4.1 Introduction	27

4.2 Current Cybersecurity Threats and Trends	27
4.3 Common Cyber Attack Vectors.....	29
4.4 Examination of Emerging Threats: The Rise of "Quishing"	31
4.5 Conclusion:	32
Chapter 5: System Design and Architecture	33
5.1 Overview of the Proposed System or Solution	34
5.2 Use Case Diagram.....	35
5.3 Use Case Descriptions	35
5.4 Justification of Design Choices and Technologies Used.....	37
5.5 Addressing Specific Cybersecurity Challenges	38
5.6 Data Flow Diagram (DFD)	39
5.7 Threat Model Diagram.....	40

Table of Figures

Figure 1:Gantt chart showing the project phases and task sequence.	10
Figure 2:Project schedule displaying task durations and start/finish dates.	11
Figure 3: VirusTotal interface showing URL scanning and its complex structure.	15
Figure 4:Google Safe Browsing interface for detecting unsafe websites.	16
Figure 5:Kaspersky QR Scanner interface.	17
Figure 6:Agile development lifecycle phases.	22
Figure 7:Four major cybersecurity threat trends: Phishing, Malware, AI-driven attacks, and Mobile-first attacks.	28
Figure 8:Comparison between standard visual-centric security warnings and accessible auditory-first security mechanisms.	30
Figure 9:The lifecycle of a Quishing (QR Phishing) attack bridging the physical and digital environments.....	31
Figure 10: Use Case Diagram.....	35
Figure 11: Data Flow Diagram (DFD).....	39
Figure 12: Threat Model Diagram	40

List of Tables

Table 1:Team Roles and Responsibilities Distribution	12
Table 2:Comparison of SafeVoice Scanner and existing solutions.	18



Chapter 1: Introduction

1.1 Introduction

The rapid digital transformation and the widespread use of the internet have brought unparalleled convenience to daily life. However, this connectivity has also introduced sophisticated cybersecurity threats. Cybercriminals frequently utilize malicious URLs and compromised Quick Response (QR) codes to execute phishing attacks, distribute malware, and steal sensitive user data. While the cybersecurity industry has developed numerous tools to detect and block these threats, the vast majority of these defensive mechanisms rely heavily on visual interfaces, such as pop-up warnings, padlock icons, or red alert screens.

This heavy reliance on visual cues creates a significant security gap and accessibility barrier for visually impaired and blind users. In an era where digital inclusivity is a global priority, this demographic remains disproportionately vulnerable to digital threats, particularly "Quishing" (QR phishing) and deceptive links. This graduation project addresses this critical issue by introducing an accessible, voice-guided cybersecurity solution tailored specifically to empower and protect visually impaired users in their digital interactions.

1.2 Problem Definition

Visually impaired individuals face immense challenges when attempting to verify the safety of web links or QR codes before interacting with them. Standard web browsers and security software communicate threats through visual warnings that screen readers may fail to interpret accurately or quickly enough before the user clicks. Furthermore, a QR code is inherently a visual pattern; a visually impaired user cannot deduce the underlying URL without scanning it, forcing them to take a leap of faith.

Consequently, these users are at a significantly higher risk of falling victim to social engineering attacks, credential theft, and malware infections. Currently, there is a lack of standalone, lightweight security tools that prioritize auditory feedback over visual alerts. The core problem this project addresses is the critical need for an accessible, independent

verification system that relies entirely on clear voice guidance to evaluate the safety of URLs and QR codes.

1.3 Proposed Solution

To mitigate the risks faced by visually impaired users, this project proposes the development of the "SafeVoice Scanner (Suspicious Link and QR Code Checker for Visually Impaired Users)". This system is an interactive, web-based platform designed with a primary focus on accessibility and robust cybersecurity. Upon accessing the application, users are greeted with an English audio prompt guiding them to either input a URL or use their device's camera to scan a QR code.

To ensure a high level of accuracy, the SafeVoice Scanner employs a Hybrid Threat Detection Model:

1. **Local Heuristic Analysis:** The Python-based backend initially analyzes the submitted link locally to identify common malicious patterns. This includes checking for IP addresses used instead of domain names, the absence of secure protocols (HTTPS), suspicious URL lengths, and the presence of deceptive characters.
2. **API Verification:** Subsequently, the link is cross-referenced in real-time with the Google Safe Browsing API to detect known malware, phishing sites, and unwanted software.

The final verdict is communicated to the user through a clear, English voice alert (e.g., "The link is safe" or "Warning, the link is dangerous"). Additionally, the entire screen background changes to Green (for safe) or Red (for dangerous) to assist users with partial vision. All scanning activities and detected threats are logged into an SQLite database for future analysis and history tracking.

1.4 System Scope

The scope of this project is confined to the development of a responsive web application optimized for both mobile devices and desktop computers. The frontend is developed using HTML, CSS, JavaScript, and the Bootstrap framework to ensure a clean and responsive user interface, utilizing the Web Speech API for voice interactions. The backend is powered by Python using the Flask framework, handling the heuristic logic and API communications. Data persistence, such as scan logs and historical threat data, is managed via a lightweight SQLite database. The project focuses exclusively on verifying URLs and QR codes and does not encompass operating system-level screen reading, file scanning, or the development of a native mobile application (iOS/Android).

1.5 Goals and Objectives

The primary aim of this project is to bridge the gap between cybersecurity and accessibility. This is achieved through the following specific objectives:

- **Enhance Digital Security:** Protect visually impaired individuals from phishing URLs and malicious QR codes.
- **Provide Full Accessibility:** Implement a 100% voice-guided user interface that requires zero visual interaction for successful operation.
- **Implement Hybrid Detection:** Combine local heuristic URL analysis with the intelligence of external security APIs (Google Safe Browsing) to maximize detection accuracy.
- **Maintain System Logs:** Utilize an SQLite database to maintain a local record of scanned links to monitor threat trends and provide users with a scan history.
- **Promote Independence:** Empower users with disabilities to navigate the internet and scan codes safely without relying on sighted assistance.

1.6 Hardware & Software Requirements

Hardware Requirements:

- A client device (smartphone, tablet, or personal computer) with an active internet connection.
- A functional integrated or external camera (required for QR code scanning).
- Audio output (Speakers/Headphones) and input (Microphone) capabilities.

Software Requirements:

- Frontend: HTML5, CSS3, JavaScript (incorporating HTML5-QRCode library and Web Speech API), and Bootstrap framework for UI components and icons.
- Backend: Python programming language with the Flask micro-framework.
- Database: SQLite (for lightweight, serverless database management).
- External Services: Google Safe Browsing API for real-time threat intelligence.
- Environment: Any modern web browser (e.g., Google Chrome, Safari, Microsoft Edge).

1.7 System Cost

This project is designed to be highly cost-effective, utilizing open-source tools and free-tier services suitable for an academic graduation project. The estimated costs in Saudi Riyals (SAR) are as follows:

- Programming IDE & Frameworks: Visual Studio Code, Python, Flask, Bootstrap, SQLite (0 SAR - Open Source).
- Threat Intelligence API: Google Safe Browsing API (0 SAR - Free tier for non-commercial/development use).
- Cloud Hosting & Domain Name: Estimated at 200 SAR (for deploying the web application online during the presentation phase).
- Internet Connection Expenses: Estimated at 300 SAR (for research, development, and system testing).
- Total Estimated Cost: 500 SAR.

1.8 Timeline

The project development lifecycle is organized into a structured academic timeline that defines the sequence of tasks and major development phases. It outlines how the work is distributed across different stages, including analysis, design, implementation, and testing. The timeline also highlights task dependencies and ensures a logical flow of activities throughout the project. The following figure presents the overall schedule of the SafeVoice Scanner system.

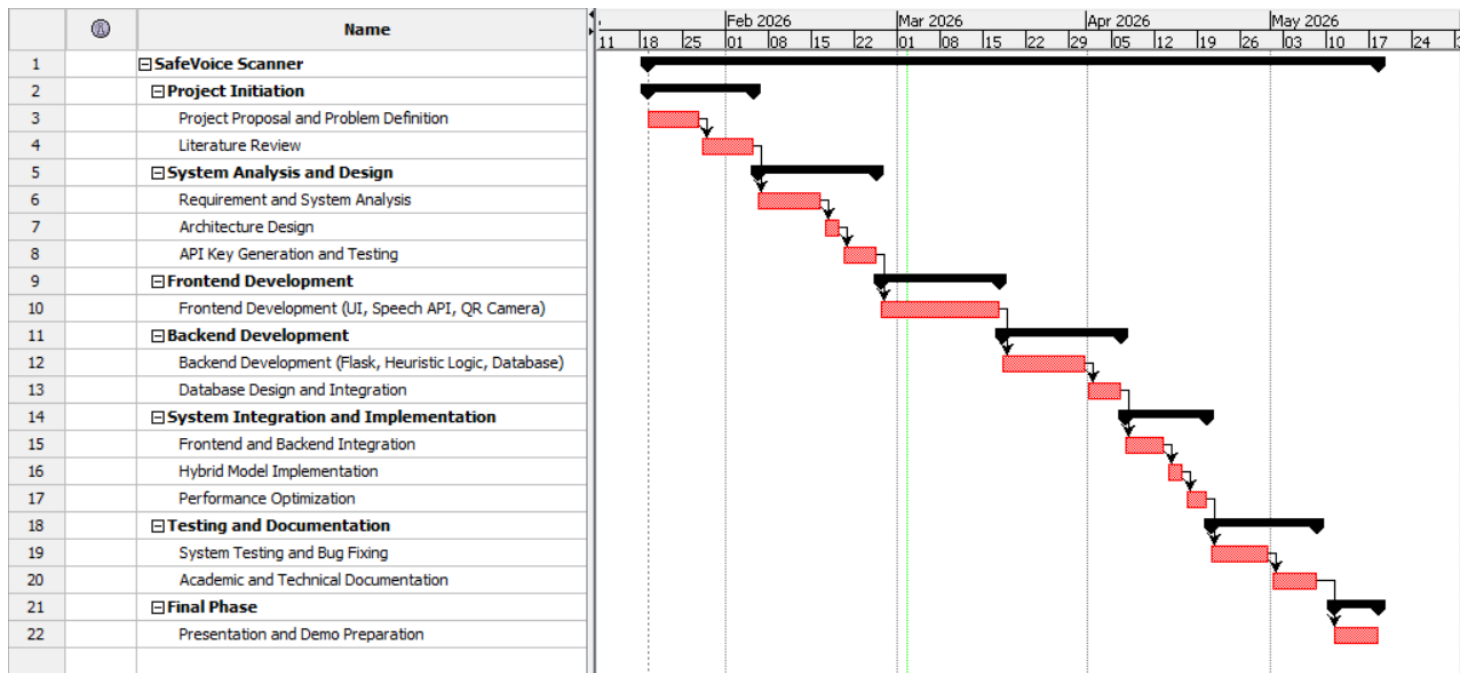


Figure 1: Gantt chart showing the project phases and task sequence.

The following figure provides a detailed breakdown of the project schedule, including task durations and timelines.

It highlights the start and finish dates for each phase, ensuring clear tracking of progress.

This structure helps in managing the development process efficiently and meeting project deadlines.

		Name	Duration	Start	Finish
1		☐ SafeVoice Scanner	86 days?	1/19/26 8:00 AM	5/18/26 5:00 PM
2		☐ Project Initiation	14 days?	1/19/26 8:00 AM	2/5/26 5:00 PM
3		Project Proposal and Problem Definition	7 days?	1/19/26 8:00 AM	1/27/26 5:00 PM
4		Literature Review	7 days?	1/28/26 8:00 AM	2/5/26 5:00 PM
5		☐ System Analysis and Design	14 days?	2/6/26 8:00 AM	2/25/26 5:00 PM
6		Requirement and System Analysis	7 days?	2/6/26 8:00 AM	2/16/26 5:00 PM
7		Architecture Design	3 days?	2/17/26 8:00 AM	2/19/26 5:00 PM
8		API Key Generation and Testing	4 days?	2/20/26 8:00 AM	2/25/26 5:00 PM
9		☐ Frontend Development	14 days?	2/26/26 8:00 AM	3/17/26 5:00 PM
10		Frontend Development (UI, Speech API, QR Camera)	14 days?	2/26/26 8:00 AM	3/17/26 5:00 PM
11		☐ Backend Development	14 days?	3/18/26 8:00 AM	4/6/26 5:00 PM
12		Backend Development (Flask, Heuristic Logic, Database)	10 days?	3/18/26 8:00 AM	3/31/26 5:00 PM
13		Database Design and Integration	4 days?	4/1/26 8:00 AM	4/6/26 5:00 PM
14		☐ System Integration and Implementation	10 days?	4/7/26 8:00 AM	4/20/26 5:00 PM
15		Frontend and Backend Integration	5 days?	4/7/26 8:00 AM	4/13/26 5:00 PM
16		Hybrid Model Implementation	3 days?	4/14/26 8:00 AM	4/16/26 5:00 PM
17		Performance Optimization	2 days?	4/17/26 8:00 AM	4/20/26 5:00 PM
18		☐ Testing and Documentation	14 days?	4/21/26 8:00 AM	5/8/26 5:00 PM
19		System Testing and Bug Fixing	8 days?	4/21/26 8:00 AM	4/30/26 5:00 PM
20		Academic and Technical Documentation	6 days?	5/1/26 8:00 AM	5/8/26 5:00 PM
21		☐ Final Phase	6 days?	5/11/26 8:00 AM	5/18/26 5:00 PM
22		Presentation and Demo Preparation	6 days?	5/11/26 8:00 AM	5/18/26 5:00 PM

Figure 2: Project schedule displaying task durations and start/finish dates.

1.9 Roles & Responsibilities

To ensure the successful completion of the project, the team adopts a highly collaborative approach where all members actively participate across all phases, including system analysis, development, testing, and report writing. The technical and documentation tasks are distributed as follows:

Table 1: Team Roles and Responsibilities Distribution

Name	Responsibilities
Munira Alajalin	Contributes to the Introduction, Goals & Objectives, and overall system methodology while coordinating the project workflow. Participates in both frontend and backend development and supports full system integration.
Turfa Aldulaimi	Works on the Problem Definition and Literature Review sections. Leads backend development using Flask, manages database design, and implements core server-side logic and testing.
Rana Aleisa	Contributes to the Proposed Solution and user analysis sections. Leads frontend development including UI/UX design, Speech API integration, QR scanning functionality, and user interaction features.
Yasmeen Alrashidi	Focuses on background research, competitive analysis, and security-related components. Responsible for API integration, heuristic detection logic, and link security analysis implementation.
Nawal Ahmed Awaji	Contributes to system scope and requirements definition. Leads quality assurance, system testing, performance evaluation, and supports final documentation and report formatting.



Chapter 2: Background & Literature Review

2.1 Introduction

This chapter provides the theoretical foundation and contextual background for the SafeVoice Scanner project. It explores the current landscape of cybersecurity threats, particularly those involving malicious URLs and Quick Response (QR) codes, and highlights how these threats disproportionately affect visually impaired individuals. Furthermore, this chapter reviews existing literature and related works, examining the current solutions available in the market. Finally, a comprehensive competitive analysis is presented to compare these existing approaches and clearly identify the unique value proposition and standout features of the proposed SafeVoice Scanner system.

2.2 Background

The internet is an indispensable tool for communication, education, and commerce. However, it is also a primary vector for cyberattacks. Two of the most prevalent attack methods are Phishing and Malware distribution via deceptive URLs.

Recently, the widespread adoption of QR codes has led to a surge in a specific type of attack known as "Quishing" (QR Phishing). Attackers embed malicious URLs within QR codes and place them in public areas or emails. When an unsuspecting user scans the code, they are directed to a fraudulent website designed to steal credentials or download malware. The fundamental challenge in combating these threats lies in the design of current cybersecurity defenses. Standard security mechanisms—such as browser warnings (e.g., Google Chrome's red "Deceptive site ahead" screen), padlock icons in the address bar, and complex antivirus reports—are heavily reliant on visual perception.

For visually impaired and blind users, these visual cues are either entirely inaccessible or extremely difficult to interpret using standard screen readers. A screen reader (like NVDA or VoiceOver) will dictate the text on the screen, but it does not proactively analyze the security of a link *before* the user interacts with it. Moreover, standard QR code scanners often automatically open the embedded link without analyzing its safety, effectively

pushing the user into a trap. This gap in accessibility leaves visually impaired users highly vulnerable, necessitating a paradigm shift towards auditory-first security solutions.

2.3 Related Work

To understand the current state of cybersecurity tools, it is essential to evaluate related systems and applications that provide similar URL or QR code scanning services. The existing solutions can be broadly categorized into three types:

1. Traditional Threat Intelligence Scanners (VirusTotal, Norton Safe Web): These are highly sophisticated web-based platforms where users can paste a URL to check it against dozens of antivirus engines. While extremely accurate, they are not designed for the visually impaired. The user interface is often cluttered with complex data, charts, and technical jargon that make navigation via a screen reader highly frustrating and time-consuming.

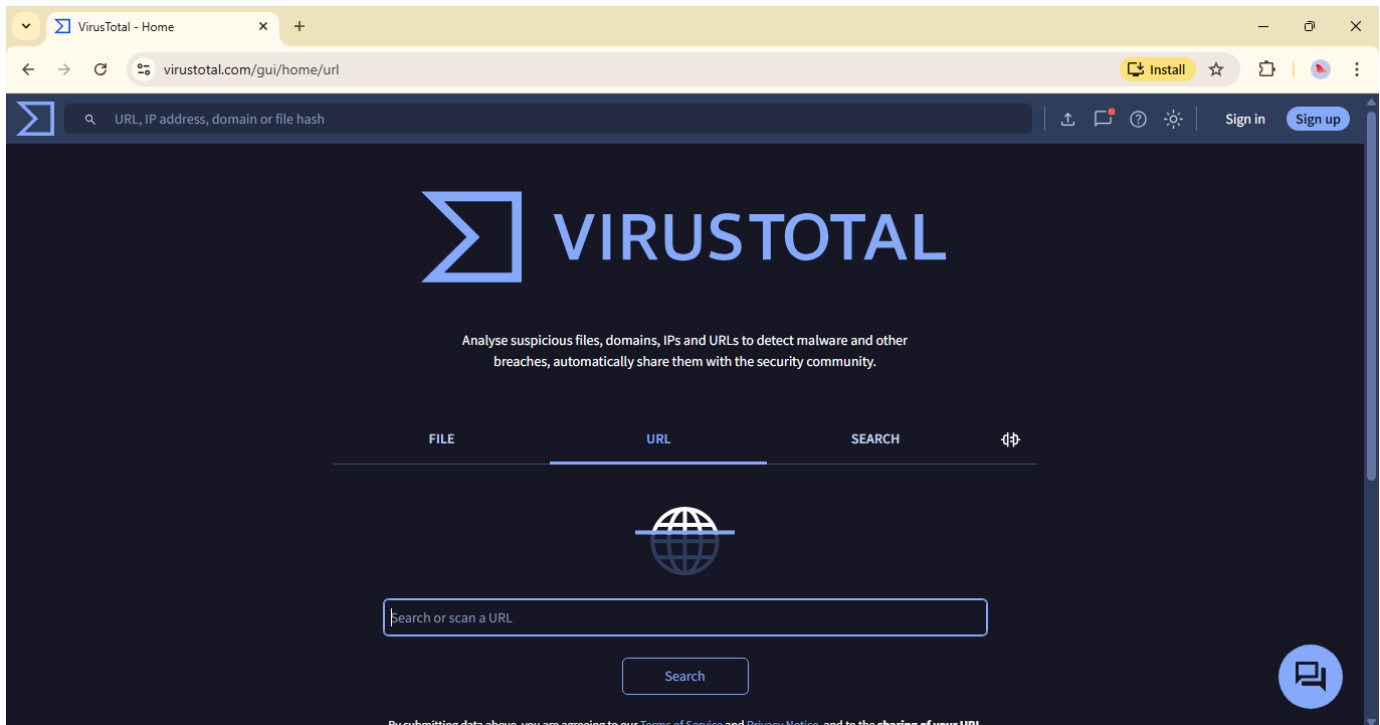


Figure 3: VirusTotal interface showing URL scanning and its complex structure.

2. Built-in Browser Security (Google Safe Browsing in Chrome/Edge): Modern browsers include built-in mechanisms to block known malicious sites. However, these systems rely on large visual warning screens. While a screen reader might eventually read the warning text, the user has already executed the action of clicking the link, which can sometimes trigger background scripts. Furthermore, these browsers do not offer a native, accessible way to scan physical QR codes safely.

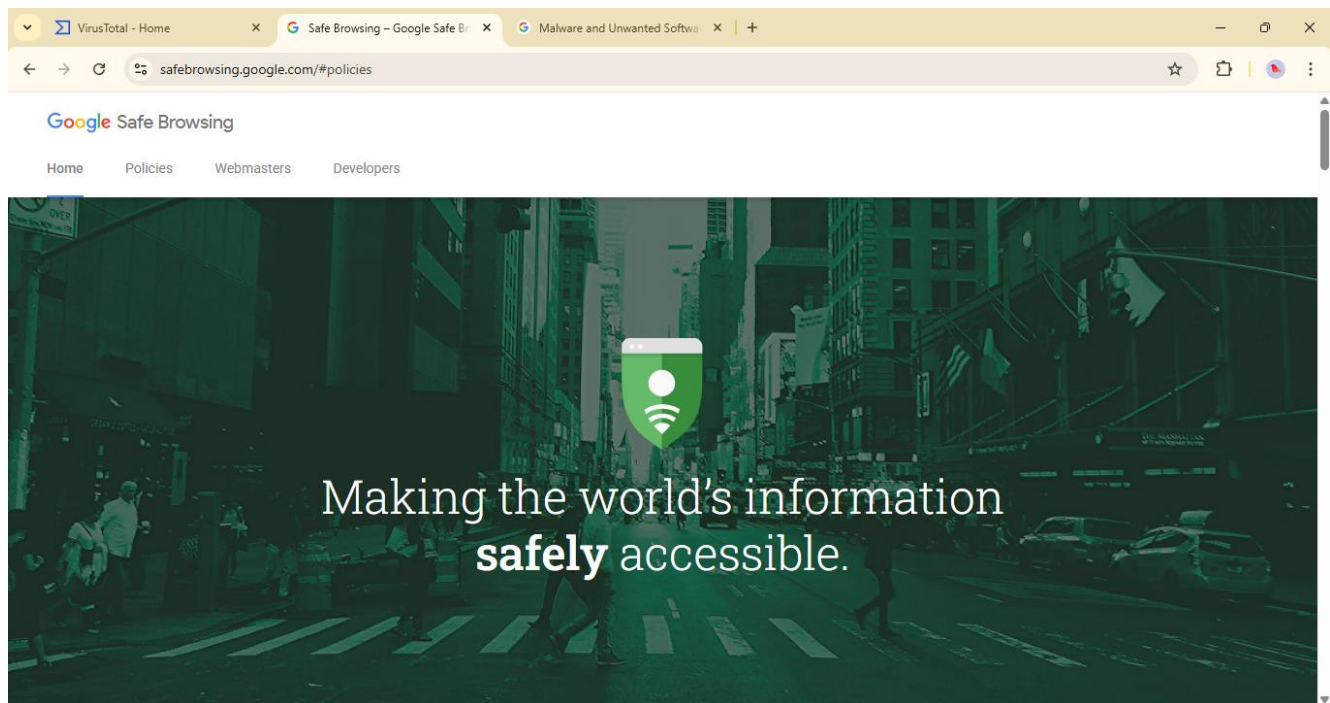


Figure 4: Google Safe Browsing interface for detecting unsafe websites.

3. Standard Mobile QR Code Scanners (Kaspersky QR Scanner): Numerous applications exist for scanning QR codes. Most of these apps focus on convenience by scanning codes and directly opening links. While Kaspersky QR Scanner provides basic security checks, it still relies on visual indicators (such as colors or icons), making it unsuitable for visually impaired users.

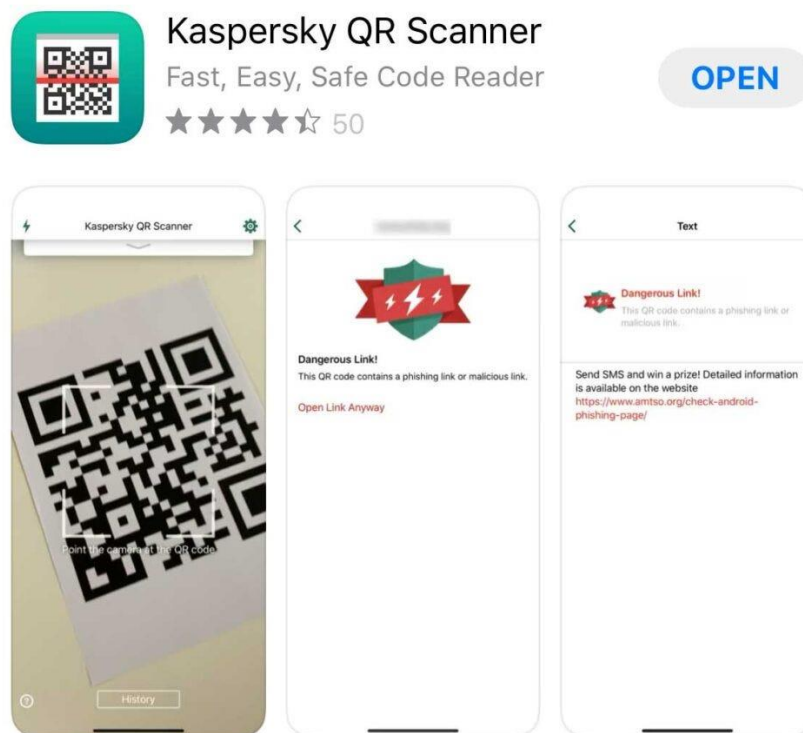


Figure 5:Kaspersky QR Scanner interface.

2.4 Competitive Analysis

To demonstrate the necessity and superiority of the SafeVoice Scanner, a competitive analysis was conducted comparing it against the primary alternative approaches: Traditional Scanners (VirusTotal), Native Browsers, and Standard QR Scanners.

Table 2: Comparison of SafeVoice Scanner and existing solutions.

Feature / Capability	SafeVoice Scanner (Proposed)	VirusTotal	Google Safe Browsing	Kaspersky QR Scanner
Primary Interface	Fully auditory, voice-guided interaction designed for hands-free use	Complex visual dashboards with technical data	Large visual warning screens	Camera-based visual interface
Accessibility for Visually Impaired Users	Excellent – built with audio-first architecture	Poor – not designed for accessibility	Limited – relies on screen readers after warning appears	Poor – highly dependent on visual cues
URL Threat Detection	Real-time hybrid analysis (Heuristic + API) before navigation	Multi-engine scan, but requires manual interpretation	Blacklist-based detection after user action	No advanced URL threat analysis
QR Code Security Verification	Secure pre-navigation QR scanning with threat analysis	Not supported (manual URL input required)	Not supported	Scans QR but often redirects immediately without deep verification
User Safety Before Navigation	High – blocks navigation until verification is complete	Low – user must analyze technical results	Moderate – warning appears after click	Low – link often opens immediately
Detection Intelligence	Hybrid detection combining local logic and external API	Multi-engine API aggregation	Internal blacklist system	Basic or minimal security filtering
Feedback Clarity	Simple binary audio feedback ("Safe" / "Dangerous")	Overwhelming technical reports	Visual alerts only	Visual indicators (icons/colors)
Targeted Design Focus	Specifically optimized for visually impaired users	Designed for cybersecurity professionals	Designed for general web users	Designed for convenience, not security

How the SafeVoice Scanner Stands Out:

Based on the competitive analysis, existing cybersecurity tools provide strong protection but fail to address accessibility for visually impaired users. The SafeVoice Scanner bridges this gap by combining security and usability in a single accessible solution. Its key advantages include:

1. **Audio-First Architecture:** The system is designed from the ground up to operate entirely through voice interaction, enabling full navigation and feedback without reliance on visual elements.
2. **Secure QR Scanning (Anti-Quishing):** The system intercepts scanned QR codes, analyzes the embedded URL using a hybrid approach (local heuristic analysis and Google Safe Browsing API), and provides a safety verdict before allowing navigation.
3. **Simplicity and Usability:** Instead of presenting complex technical data, the system delivers clear and concise audio feedback ("Safe" or "Dangerous"), ensuring ease of use and practical accessibility for daily usage.
4. **Pre-Interaction Threat Prevention:** The system evaluates links and QR codes before the user interacts with them, reducing the risk of accidental exposure to malicious content.



Chapter 3: System Methodology & Analysis

3.1 Introduction

This chapter outlines the methodological approach and system analysis for the development of the SafeVoice Scanner. A well-defined methodology is crucial to ensure that the system meets its primary goal: providing an accessible, auditory-first cybersecurity tool for visually impaired users. This chapter details the chosen software development methodology, describes the characteristics of the target users, identifies their specific requirements, and comprehensively lists both the functional and non-functional software requirements and specifications necessary to build the proposed system.

3.2 System Methodology

For the development of the SafeVoice Scanner, the Agile Software Development Methodology has been adopted. Agile is highly iterative and flexible, making it the ideal choice for a project focused on accessibility and user experience (UX).

Given that the target audience consists of visually impaired users, their interaction with the system relies heavily on auditory feedback and simple navigation. The Agile methodology allows the development team to build the system in small, manageable increments (sprints). For instance, the voice-guidance module can be developed and tested in one sprint, while the hybrid threat detection (Heuristic + API) can be integrated in the next. This iterative process allows for continuous testing, immediate feedback integration, and the flexibility to adjust voice prompts, scanning speeds, or UI elements without disrupting the entire development lifecycle.

Why Agile is the Best Fit for This Project:

- **User-Centric Flexibility:** Developing a tool for visually impaired users requires continuous tuning of voice prompts, screen colors, and response times. Agile allows the team to adapt and refine these elements iteratively based on ongoing testing, ensuring maximum accessibility.
- **Incremental Development (Sprints):** The SafeVoice Scanner involves distinct technical modules (Bootstrap UI, Web Speech API, Python backend, Google API

- integration). Agile enables the team to complete, test, and perfect one functional module before moving to the next.
- **Continuous Testing:** In a cybersecurity application, ensuring that the hybrid detection model correctly identifies threats is critical. Agile facilitates continuous integration and testing, allowing the team to identify bugs or false positives early in the development cycle.
 - **High Adaptability:** If a specific technology (e.g., the JavaScript QR code library) proves incompatible or inaccessible during development, Agile provides the structural framework to quickly pivot to an alternative solution without restarting the entire project.



Figure 6:Agile development lifecycle phases.

3.3 User Characteristics

The system is designed with a highly specific demographic in mind. Understanding the characteristics of these users is fundamental to the system's success:

- **Primary Users (Visually Impaired and Blind Individuals):** These users have little to no visual perception. They rely heavily on auditory information, screen readers, and tactile feedback to navigate digital environments. They require applications with straightforward navigation, zero clutter, and clear auditory cues. Their technical expertise may vary from beginner to advanced, necessitating a system that is intuitive and requires no complex setup or technical jargon.
- **Users with Low Vision / Color Blindness:** These users retain some visual capabilities but struggle to read small text or identify complex visual warnings. They benefit from high-contrast interfaces and full-screen color indicators (such as a solid Green or Red screen) combined with audio.
- **Secondary Users (General Public):** Sighted users who prefer a hands-free, voice-guided experience to quickly scan QR codes or check URLs on the go without reading through complex threat reports.

3.4 User Requirements

Based on the characteristics of the target demographic, the specific user requirements for the SafeVoice Scanner are identified as follows:

1. **Auditory Independence:** The user must be able to operate the entire system—from opening the website to receiving the final security verdict—relying solely on English voice guidance without needing sighted assistance.
2. **Simplicity of Action:** The system must minimize user inputs. The user should only be required to make a simple choice (scan QR or input URL) and let the system handle the rest automatically.

3. **Clear and Unambiguous Feedback:** The user must receive the final security result in simple, definitive terms (e.g., "The link is safe" or "Dangerous") rather than complex security logs or probability percentages.
4. **Fast Processing:** Since blind users cannot see a visual "loading bar," the system must process the URL and deliver the audio result quickly to prevent anxiety or confusion regarding whether the system has frozen.

3.5 Software Requirements and Specifications

To fulfill the user requirements and achieve the project's goals, the system specifications are divided into Functional and Non-Functional Requirements.

a- Functional Requirements

Functional requirements define the specific behaviors, tasks, and functions the SafeVoice Scanner must perform:

- FR1: Voice Guidance and Prompts: The system must automatically trigger a welcome audio message using the Web Speech API upon loading, instructing the user on how to interact with the platform.
- FR2: QR Code Scanning: The system must be able to request permission and access the device's camera to scan physical QR codes and extract the embedded URL automatically.
- FR3: Manual URL Input: The system must provide an accessible option for users to paste or type a URL manually for verification.
- FR4: Hybrid Threat Detection:
 - FR4.1 (Local Heuristics): The Python backend must analyze the URL locally to detect phishing indicators (e.g., IP addresses instead of domains, lack of HTTPS protocol, and suspicious symbols).
 - FR4.2 (API Verification): The backend must send the extracted or inputted URL to the Google Safe Browsing API to check against known malware and phishing databases.

- FR5: Result Generation & Audio Output: The system must evaluate the combined results from the hybrid detection and output a clear English audio alert announcing the final status (Safe or Dangerous).
- FR6: Visual Color Shift: The system must change the entire background color of the interface to Green if the link is safe, and Red if the link is malicious, accompanying the audio alert.
- FR7: Scan Logging: The system must record the details of each scan (URL, Timestamp, Detection Result) into a local SQLite database for historical tracking.

b- Non-Functional Requirements

Non-functional requirements specify the quality attributes, performance, and operational constraints of the system:

- NFR1: Usability and Accessibility: This is the highest priority. The system interface must comply with basic web accessibility guidelines. It must be completely operable without visual reliance, ensuring high usability for the blind.
- NFR2: Performance and Response Time: The threat analysis process (from the moment the QR is scanned or the URL is submitted to the moment the audio result is played) should ideally be completed within 3 to 5 seconds to ensure a seamless experience.
- NFR3: Cross-Platform Compatibility: The web application must function correctly on major modern web browsers (Google Chrome, Safari, Microsoft Edge) across different operating systems (Windows, macOS, iOS, Android).
- NFR4: Security and Privacy: The system must not store personal user data. The communication between the frontend, the Python Flask backend, and the Google Safe Browsing API must be secure.
- NFR5: Reliability: The system should handle API timeouts or camera access denials gracefully, providing appropriate audio error messages (e.g., "Camera access denied" or "Unable to connect to the server") instead of crashing silently.



Chapter 4: Cybersecurity Threat Landscape



4.1 Introduction

This chapter studies the current cybersecurity threat landscape, providing an in-depth analysis of the risks present in the modern digital environment. As technology advances and internet access becomes increasingly ubiquitous, cybercriminals continuously evolve their tactics. There has been a noticeable shift from purely technical exploits such as breaching complex firewalls to sophisticated attacks that manipulate human psychology and exploit accessibility gaps in everyday software.

Understanding this landscape is the critical first step in developing effective countermeasures. This chapter explores various general threats and trends, such as malware distribution, automated attacks, and social engineering. However, it specifically narrows its focus on the attack vectors most relevant to this project: malicious URLs, compromised QR codes, and the critical, often-overlooked vulnerabilities faced by visually impaired users who interact with visually-centric security systems.

4.2 Current Cybersecurity Threats and Trends

The contemporary cybersecurity landscape is characterized by a significant rise in deceptive attacks targeting the end-user. Rather than attempting to compromise secure corporate networks, modern attackers find it much more efficient and cost-effective to trick individuals into compromising themselves. Currently, the most prominent trends defining this landscape include:

- **Phishing and Credential Harvesting:** Attackers create highly convincing deceptive websites that mimic legitimate institutions, such as banking portals, university login pages, or popular social media platforms. The goal is to trick users into voluntarily submitting their sensitive login credentials or financial information.
- **Malware and Ransomware Distribution:** Cybercriminals frequently use deceptive web links to silently download malicious software onto a victim's device. Once installed, this software can encrypt personal files to demand a ransom, deploy

- spyware to steal data in the background, or enslave the device into a botnet for further illicit activities.
- **The Integration of AI in Cyberattacks:** Attackers are now utilizing Artificial Intelligence (AI) to generate flawless, grammatically correct phishing emails and highly personalized scam messages at scale. This makes identifying fraudulent content much harder for the average user, as the traditional red flags (like poor spelling) are no longer reliable.
 - **Mobile-First Attacks:** With the global shift towards smartphone dominance, attackers are increasingly tailoring their malicious links and deceptive tactics to target mobile users. Mobile browser interfaces typically obscure full URLs to save screen space, making it significantly easier to trick users into clicking fraudulent links that they might have otherwise inspected on a desktop.



Figure 7: Four major cybersecurity threat trends: Phishing, Malware, AI-driven attacks, and Mobile-first attacks.

4.3 Common Cyber Attack Vectors

An attack vector is the specific path, method, or scenario by which a hacker gains unauthorized access to a computer, network, or user device to deliver a payload or achieve a malicious outcome. In the context of this project, the most critical attack vectors and systemic vulnerabilities include the following:

4.3.1 Malicious URLs (The Primary Vector)

URLs are the primary navigation tool of the internet, making them the most common vehicle for delivering cyber threats. Cybercriminals employ various sophisticated evasion techniques to disguise harmful URLs and bypass basic security filters. This includes:

- **URL Shorteners:** Utilizing legitimate services like bit.ly or tinyurl to completely mask the true destination of the link. The user cannot see the endpoint until they have already clicked and navigated to it.
- **Typosquatting and Homograph Attacks:** Typosquatting relies on common spelling mistakes (e.g., "faceb00k.com" instead of "facebook.com"). Homograph attacks are even more deceptive, employing visually similar characters from different alphabets to spoof legitimate domains (e.g., using a Cyrillic "a" in "amazon.com").
- **Direct IP Routing:** Utilizing raw IP addresses instead of domain names (e.g., <http://192.168.1.55/login>) to bypass standard DNS-based security blocklists and hide the attacker's identity.

4.3.2 Social Engineering

This vector relies purely on psychological manipulation rather than technical hacking. Attackers create scenarios that induce a false sense of urgency, fear, or overwhelming curiosity. For example, a user might receive a fake SMS alert stating, "Your bank account has been compromised, click here immediately to verify your identity." This tactic bypasses rational thinking, prompting the user to click a malicious link rapidly without pausing to evaluate its authenticity.

4.3.3 The Visual-Centric Security Vulnerability

While the aforementioned vectors affect all demographics, there is a systemic architectural vulnerability in modern software that disproportionately affects individuals with disabilities: standard cybersecurity defenses are overwhelmingly visual. Modern web browsers, operating systems, and antivirus software rely heavily on graphical indicators such as padlock icons, color-coded trust signals (green for safe, red for danger), complex text-based URL analysis, and sudden pop-up warning screens.



Figure 8: Comparison between standard visual-centric security warnings and accessible auditory-first security mechanisms.

For visually impaired individuals, this heavy reliance on visual cues creates a critical and dangerous security blind spot. Standard screen readers (like Apple's VoiceOver, Android's TalkBack, or NVDA) process web pages linearly and rely on proper coding (such as ARIA tags). When a sudden visual warning pops up, the screen reader may not immediately interpret or announce it. In many cases, it fails to read the warning entirely before a malicious background script is already executed.

Furthermore, tasks that sighted users perform instantly such as scanning the exact spelling of a URL to detect a typosquatting attack are incredibly tedious, time-consuming, and error-prone when relying on auditory spelling letter-by-letter. Consequently, visually

impaired users are forced to rely on blind trust rather than active verification. This massive accessibility gap is actively exploited by cybercriminals, highlighting the urgent and undeniable need for auditory-first security mechanisms.

4.4 Examination of Emerging Threats: The Rise of "Quishing"

One of the most rapidly emerging and challenging threats in the cybersecurity field is "Quishing," a portmanteau of QR Code and Phishing. As Quick Response (QR) codes have become ubiquitous worldwide for accessing restaurant menus, making payments, and sharing contact information, cybercriminals have quickly adapted to weaponize them.

4.4.1 The Quishing Mechanism

Unlike traditional phishing which usually occurs via email or SMS, Quishing bridges the physical and digital worlds. An attacker places a physical sticker of a malicious QR code over a legitimate one in a public space (such as a parking meter, an advertising billboard, or a retail store counter). Because QR codes are machine-readable matrices of squares, standard corporate email security filters, firewalls, and human eyes cannot analyze their contents. They easily bypass traditional digital defenses because the threat originates in the physical environment.



Figure 9: The lifecycle of a Quishing (QR Phishing) attack bridging the physical and digital environments.

4.4.2 The Threat to the Visually Impaired

While Quishing is undoubtedly dangerous for sighted users, it represents a severe, unmitigated threat to the blind and visually impaired community. A QR code completely abstracts and hides the destination URL. A user has absolutely no contextual clues regarding where the code leads until the device scans it.

Compounding this danger is the behavior of standard QR scanning applications, which are designed for speed and convenience. Most native camera apps will execute the link or open the browser immediately upon successful scanning. Without an auditory intervention, an accessible preview, or a pre-scan safety check, a visually impaired user is forced to trust the physical environment blindly. This makes them prime, highly vulnerable targets for this sophisticated and emerging attack vector.

4.5 Conclusion:

The evolving nature of cybersecurity threats clearly demonstrates that traditional, visual-based warnings are no longer sufficient to protect all demographics in the modern digital age. The attack vectors of malicious URLs, social engineering, and Quishing explicitly exploit the gap between visual security cues and auditory accessibility. By intercepting these specific attack vectors and applying a Hybrid Threat Detection model before any browser navigation or script execution occurs, the SafeVoice Scanner effectively mitigates these risks. It provides a robust, auditory-first defense mechanism specifically tailored to secure the digital independence of visually impaired users.



Chapter 5: System Design and Architecture

5.1 Overview of the Proposed System or Solution

The proposed system, SafeVoice Scanner, is an innovative and accessible cybersecurity solution specifically designed to protect visually impaired users from malicious URLs and QR code-based attacks. The system aims to bridge the gap between digital security and accessibility by providing a fully voice-guided platform that allows users to verify the safety of links before interacting with them.

SafeVoice Scanner operates as a web-based application that integrates both usability and security into a single streamlined solution. Upon accessing the system, users are guided through clear audio instructions that enable them to either input a URL manually or scan a QR code using their device's camera. This voice-first interaction ensures that users can navigate and utilize the system independently without relying on visual cues.

To deliver reliable and accurate threat detection, the system adopts a hybrid analysis approach. It first performs a local heuristic evaluation of the provided URL to identify suspicious patterns such as unusual structure, absence of secure protocols, or deceptive formatting. Afterward, the system enhances its analysis by verifying the link against trusted external threat intelligence services, ensuring detection of known phishing sites, malware, and unsafe web content.

The outcome of the analysis is communicated to the user through simple and clear auditory feedback, such as indicating whether the link is safe or dangerous. This approach eliminates the complexity found in traditional cybersecurity tools and replaces it with straightforward, user-friendly communication. Additionally, visual indicators such as color changes (green for safe and red for unsafe) are included to support users with partial vision.

The system also maintains a record of scanned links and their results, allowing for tracking and analysis of previous interactions. Overall, SafeVoice Scanner provides a proactive and

accessible defense mechanism that empowers visually impaired users to safely navigate digital environments while reducing their exposure to cyber threats.

5.2 Use Case Diagram

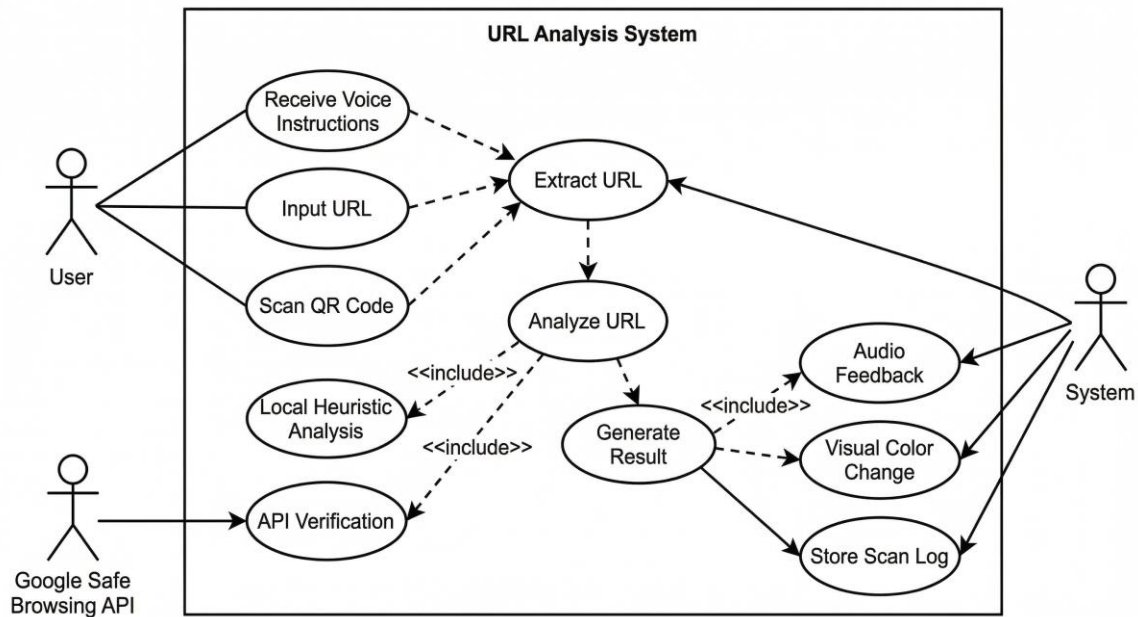


Figure 10: Use Case Diagram

5.3 Use Case Descriptions

	Use Case	Actor	Description	Precondition	Postcondition
1	Receive Voice Instructions	User	When the system starts, it provides voice guidance to help the user understand how to use the application.	System is loaded	User is ready to interact
2	Input URL	User	The user manually enters or pastes a	User chooses URL input option	URL is sent for analysis

			URL into the system for verification.		
3	Scan QR Code	User	The user scans a QR code using the device camera to extract the embedded URL.	Camera permission is granted	URL is extracted
4	Extract URL	System	The system extracts the URL from the scanned QR code.	QR code is scanned successfully	URL is ready for analysis
5	Analyze URL	System	The system analyzes the URL using both local heuristics and external API verification.	URL is available	Threat evaluation is completed
6	Local Heuristic Analysis	System	The system checks for suspicious patterns such as: use of IP address instead of domain, missing HTTPS, unusual length or characters.	URL received	Preliminary risk indicators identified
7	API Verification	Google Safe Browsing API	The system sends the URL to an external API to check against known threats.	Internet connection available	API response received
8	Generate Result	System	The system combines results from heuristic and API analysis to	Analysis completed	Final decision generated

			determine if the link is safe or dangerous.		
9	Audio Feedback	System	The system announces the result using voice output (e.g., "Safe" or "Dangerous").	Result available	User hears the result
10	Visual Color Change	System	The system changes the screen color: Green → Safe, Red → Dangerous.	Result available	Visual feedback displayed
11	Store Scan Log	System	The system saves scan details (URL, timestamp, result) in the database.	Scan completed	Data stored for history

5.4 Justification of Design Choices and Technologies Used

The design choices and technologies used in the SafeVoice Scanner system were carefully selected to align with both accessibility requirements and cybersecurity effectiveness. The primary focus was to ensure that the system remains lightweight, responsive, and fully accessible to visually impaired users while maintaining reliable threat detection capabilities.

The decision to develop the system as a web-based application was driven by the need for cross-platform accessibility. By utilizing standard web technologies such as HTML5, CSS3, JavaScript, and Bootstrap, the system can operate seamlessly across various devices, including smartphones, tablets, and desktop computers, without requiring installation. This significantly enhances usability and reach.



The integration of the Web Speech API plays a crucial role in enabling voice-guided interaction. This technology allows the system to provide real-time auditory feedback, ensuring that users can navigate and operate the application independently without relying on visual elements.

On the backend, Python with the Flask framework was chosen due to its simplicity, flexibility, and strong support for rapid development. Flask enables efficient handling of HTTP requests, integration with external APIs, and implementation of custom heuristic logic for URL analysis.

The use of a Hybrid Threat Detection Model combining local heuristic analysis and the Google Safe Browsing API ensures both speed and accuracy. Local heuristics provide immediate preliminary analysis, while the API offers reliable verification against a global database of known threats.

Finally, SQLite was selected as the database solution due to its lightweight nature and ease of integration. It allows efficient storage of scan logs without requiring complex server configurations, making it ideal for an academic project.

5.5 Addressing Specific Cybersecurity Challenges

The SafeVoice Scanner system is specifically designed to address several critical cybersecurity challenges that disproportionately affect visually impaired users.

One of the primary challenges is the inability to interpret visual security warnings. Traditional systems rely heavily on graphical indicators such as warning screens, icons, and color signals. SafeVoice Scanner eliminates this barrier by providing clear auditory feedback, ensuring that users receive immediate and understandable security information.

Another challenge is the risk posed by malicious URLs and phishing attacks. The system mitigates this by analyzing URLs before any interaction occurs. Through heuristic checks

and API verification, it identifies suspicious patterns and known threats, preventing users from accessing harmful websites.

The system also addresses the growing threat of QR-based attacks (Quishing). Unlike conventional QR scanners that automatically open links, SafeVoice Scanner intercepts the URL, evaluates its safety, and informs the user before any action is taken.

Additionally, the system tackles the issue of user uncertainty and lack of technical knowledge. By simplifying the output into clear binary results (“Safe” or “Dangerous”), it removes the need for users to interpret complex security reports.

Overall, the system provides a proactive, accessible, and user-centered approach to cybersecurity, reducing the risk of exploitation and enhancing digital independence.

5.6 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) illustrates how data moves through the SafeVoice Scanner system and how different components interact with each other.

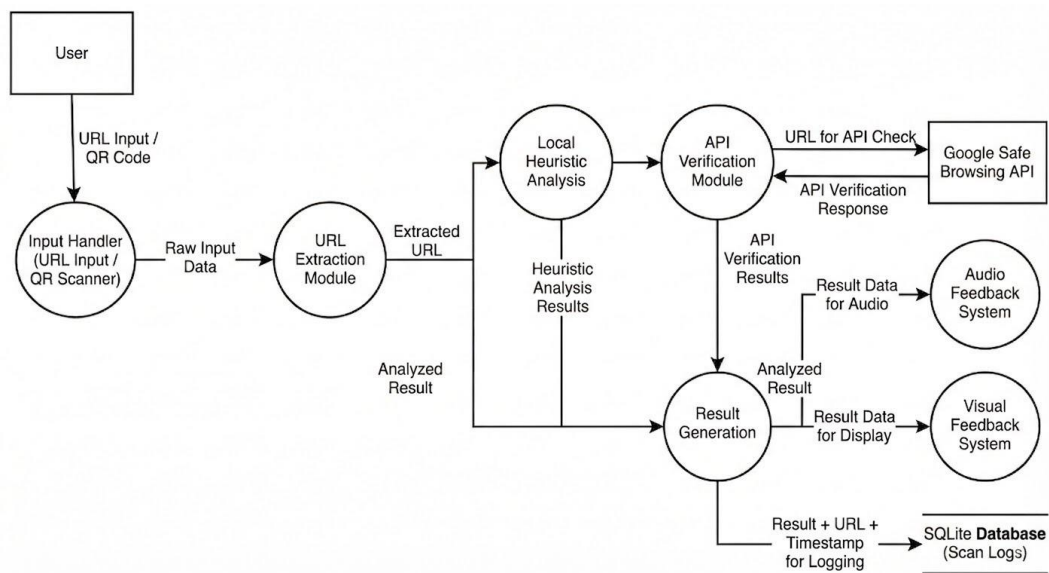


Figure 11: Data Flow Diagram (DFD)

5.7 Threat Model Diagram

The Threat Model Diagram identifies potential security risks within the SafeVoice Scanner system and outlines how these threats are mitigated.

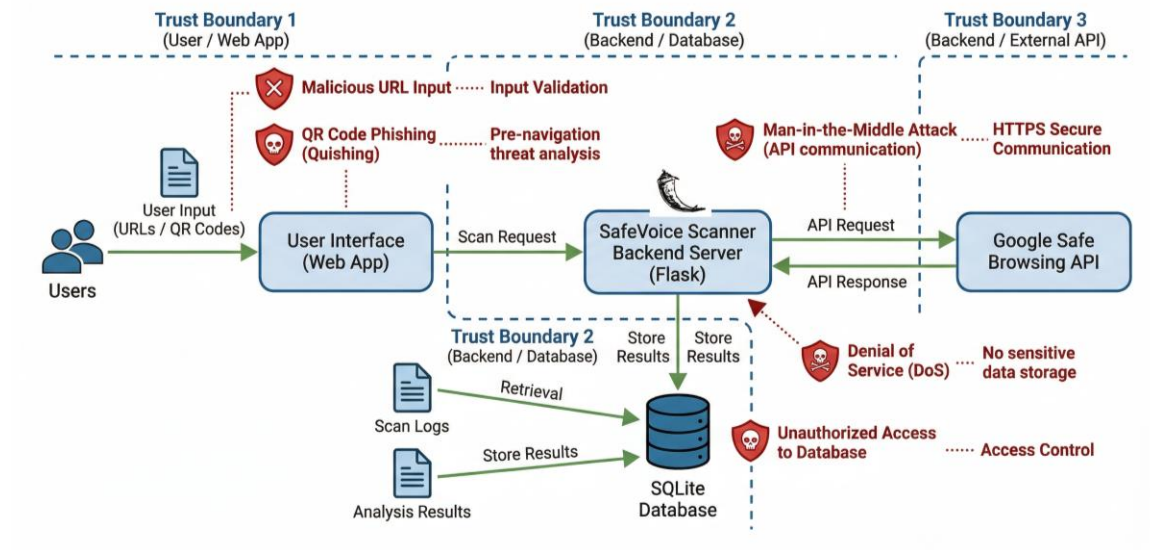


Figure 12: Threat Model Diagram